Biểu diễn token bằng mạng nơ-ron nông

One-hot, embedding, CBOW và Skip-gram

Tài liệu học tập và thực nghiệm từ notebook của dự án

08/10/2026

Tóm tắt

Báo cáo giải thích vì sao token cần được biến thành vector số và cách mạng nơ-ron nông học embedding từ ngữ cảnh. Hai nhiệm vụ Word2Vec được xét riêng: CBOW dự đoán từ trung tâm từ các từ xung quanh, còn Skip-gram dự đoán ngữ cảnh từ từ trung tâm. Công thức truyền xuôi, cross-entropy, gradient và kết quả NumPy của dự án được trình bày kèm giới hạn của corpus tổng hợp nhỏ.

Mục lục

1 Từ văn bản đến vector **1**

2 CBOW: ngữ cảnh dự đoán từ giữa **2**

2.1 Mẫu huấn luyện và truyền xuôi 2

2.2 Loss và đạo hàm 2

3 Skip-gram: từ giữa dự đoán ngữ cảnh **3**

4 Thiết kế dữ liệu và quy trình huấn luyện **3**

5 Kết quả và cách diễn giải **3**

6 Sử dụng embedding và các giới hạn **4**

7 Tái lập và câu hỏi tự kiểm tra **4**

1 Từ văn bản đến vector

Máy tính chỉ xử lý số, nên trước khi học cần định nghĩa token và từ vựng. Trong notebook, mỗi từ như `học_sinh` hay `tài_liệu` là một token đã được chuẩn bị. Đánh số token từ 0 đến $V-1$ chỉ tạo ID; chênh lệch hai ID không biểu diễn độ khác nhau về nghĩa.

Cách biểu diễn	Định nghĩa ngắn	Giới hạn quan trọng
ID và one-hot	Vector dài V, chỉ một phần tử bằng 1	Các vector từ khác nhau trực giao, không mã hóa gần nghĩa
Count vector	Đếm số lần token xuất hiện	Không trực tiếp biểu diễn thứ tự hay ngữ cảnh cục bộ
TF–IDF	Cân tần suất theo mức phổ biến toàn corpus	Là đặc trưng thống kê, không tự học quan hệ dự đoán
Embedding	Tra một hàng trong ma trận $W_{in} \in \mathbb{R}^{V \times d}$	Ý nghĩa phụ thuộc corpus và mục tiêu huấn luyện

Bảng 1: Các cách đưa token vào mô hình; phạm vi bám theo slide môn học trang 6–9.

Với token có ID i và one-hot e_i, phép nhân $e_i W_{in} = W_{in}[i]$ chính là tra hàng embedding của token đó. Thông thường $d \ll V$, nhưng embedding không bắt buộc có ít chiều hơn một biểu diễn đầu vào khác. Token hóa cũng quyết định từ vựng: nếu một từ mới không có ID và không có cơ chế `<UNK>` hay subword, mô hình không nhận được từ đó.

2 CBOW: ngữ cảnh dự đoán từ giữa

2.1 Mẫu huấn luyện và truyền xuôi

Ví dụ "học_sinh [đọc] sách" có ngữ cảnh là hai từ hai bên và nhãn là "đọc". Gọi a, b là ID hai từ ngữ cảnh. Với một mẫu:

$$X = \frac{e_a + e_b}{2} \in \mathbb{R}^V, \tag{1}$$

$$H = XW_{in} = \frac{W_{in}[a] + W_{in}[b]}{2} \in \mathbb{R}^d, \tag{2}$$

$$S = HW_{out} + b_{out} \in \mathbb{R}^V, \tag{3}$$

$$P_j = \frac{\exp(S_j)}{\sum_{k=1}^{V} \exp(S_k)}. \tag{4}$$

$W_{out} \in \mathbb{R}^{d \times V}$ và $b_{out} \in \mathbb{R}^V$. Trong cách nhìn của bài học, W_{in} tạo biểu diễn ẩn, còn W_{out} biến biểu diễn thành phân phối từ dự đoán. Đây là mạng hai phép chiếu với một tầng ẩn tuyến tính, không có ReLU hay tanh ở giữa. Softmax nằm ở tầng đầu ra. Lấy trung bình khiến thứ tự hai từ ngữ cảnh bị mất.

2.2 Loss và đạo hàm

Với B mẫu và nhãn one-hot Y, cross-entropy trung bình là

$$L = -\frac{1}{B} \sum_{i=1}^{B} \log P_{i,y_i}. \tag{5}$$

Đặt $G = (P - Y)/B$. Khi đó

$$\frac{\partial L}{\partial W_{out}} = H^\top G, \qquad \frac{\partial L}{\partial b_{out}} = \sum_i G_{i,:}, \qquad \frac{\partial L}{\partial W_{in}} = X^\top (G W_{out}^\top). \tag{6}$$

Gradient của W_{in} chứng tỏ embedding được học thông qua lỗi dự đoán từ trung tâm. Khi cài đặt softmax, trừ giá trị logit lớn nhất trước khi lấy exp để tránh tràn số; phép trừ chung này không đổi phân phối.

3 Skip-gram: từ giữa dự đoán ngữ cảnh

Skip-gram đổi vai trò đầu vào và nhãn. Với chuỗi "học_sinh đọc sách" và cửa sổ bán kính 1, có các cặp (học_sinh → đọc), (đọc → học_sinh), (đọc → sách), (sách → đọc). Mỗi cặp được xem như một mẫu dự đoán riêng.

Ở một cặp có từ trung tâm c và từ ngữ cảnh o, đặt $X = e_c$, $H = XW_{in} = W_{in}[c]$, $S = HW_{out} + b_{out}$ và $L = -\log P(o \mid c)$. Công thức gradient giống CBOW vì lớp tính toán không đổi; thứ khác là cách tạo X và y. Một từ trung tâm có thể đi với nhiều ngữ cảnh, nên xác suất được phân bố trên nhiều nhãn. Đầu ra Skip-gram là phân phối *từ ngữ cảnh*, không phải một câu đã giải mã.

CBOW và Skip-gram trong notebook dùng full softmax vì từ vựng nhỏ. Với từ vựng lớn, chi phí tính phân phối trên toàn bộ V từ có thể cao; các biến thể Word2Vec dùng hierarchical softmax và negative sampling [1, 2]. Thay negative sampling sẽ thay cả hàm mục tiêu và gradient, không đơn thuần là bỏ bớt vài phần tử softmax.

4 Thiết kế dữ liệu và quy trình huấn luyện

Notebook tạo ba nhóm câu ngắn tổng hợp thuộc học tập, động vật và lập trình. Mỗi tổ hợp chủ ngữ–tân ngữ đi cùng hai động từ hợp lệ, ví dụ "đọc" hoặc "xem". Các cặp ngữ cảnh được phân giữa train/validation/test, mỗi cặp và hai biến thể động từ luôn ở cùng một tập. Từ vựng gồm $V = 24$ token, embedding có $d = 4$ chiều; số tham số $24 \cdot 4 + 4 \cdot 24 + 24 = 216$.

CBOW có 30 câu train, 12 validation và 12 test. Skip-gram tạo cửa sổ bán kính 1 từ các câu ấy, cho 120 cặp train, 48 validation và 48 test. Cả hai dùng gradient descent trên batch đầy đủ và chọn epoch bằng validation, sau đó mới tính test. Hàm `gradient_check` so đạo hàm giải tích của tất cả 216 tham số với sai phân hữu hạn; sai số chuẩn hóa lớn nhất được lưu là 4.97×10^{-10}.

5 Kết quả và cách diễn giải

Mô hình	Test CE	Baseline unigram CE	Test top-1
CBOW	0.7008	2.1972	50.0%
Skip-gram	1.4443	3.1790	27.1%

Bảng 2: Kết quả đã lưu trong notebook với seed 42 trên corpus tổng hợp. CE là cross-entropy trung bình theo mẫu/cặp của từng nhiệm vụ.

CBOW có nhiều ngữ cảnh giống hệt nhau nhưng hai nhãn cân bằng. Dự đoán gần 0.5–0.5 cho hai động từ là hợp lý; cross-entropy lý tưởng của riêng hai lựa chọn gần $\ln 2 \approx 0.693$, trong khi top-1 chỉ đúng khoảng một nửa. Ví dụ notebook cho cặp "sinh_viên [?] tài_liệu" dự đoán "đọc" khoảng 0.505 và "xem" khoảng 0.493.

Với Skip-gram, tuy câu test khác câu train, *toàn bộ 36 cặp từ khác nhau* xuất hiện ở test cũng đã xuất hiện ở train. Vì vậy, kết quả test không chứng minh khả năng xử lý cặp từ hoàn toàn mới. Hai giá trị CE của CBOW và Skip-gram cũng không so sánh trực tiếp để kết luận kiến trúc nào tốt hơn, vì số mẫu, đầu vào, nhãn và độ bất định khác nhau.

6 Sử dụng embedding và các giới hạn

Sau huấn luyện, có thể lấy các hàng của W_{in} làm embedding. Một phép kiểm tra gần nhau là cosine

$$\cos(u, v) = \frac{u^\top v}{\|u\|_2 \|v\|_2}. \tag{7}$$

Đây là phép đo khoảng cách giữa vector đã học, khác với phép dự đoán $\text{softmax}(zW_{out} + b_{out})$ của decoder. Notebook thấy "xem" gần "đọc" theo cosine trong corpus đồ chơi; một quan sát như vậy không đủ để khẳng định model hiểu ngữ nghĩa tiếng Việt nói chung. Hình PCA hai chiều chỉ là phép chiếu để xem, không thay đổi mô hình và có thể làm méo quan hệ khoảng cách.

Các embedding kiểu Word2Vec lưu một vector cố định cho mỗi token, nên từ đa nghĩa có thể bị gộp nghĩa. Corpus nhỏ làm số ngữ cảnh rất hạn chế; không có subword hay cơ chế xử lý từ ngoài từ vựng. Muốn đánh giá trên tiếng Việt thật cần corpus được phép dùng, token hóa phù hợp, tách dữ liệu không rò rỉ và cả đánh giá theo nhiệm vụ sử dụng embedding.

7 Tái lập và câu hỏi tự kiểm tra

Chạy `encoder_decoder_minh_hoa.ipynb` từ đầu với các thư viện trong `requirements.txt`. Notebook ghi rõ phiên bản Python/NumPy, cố định seed và xuất các chỉ số ở trên. Để khảo sát kích thước embedding, thay d nhưng giữ cách chia và seed, chọn cấu hình theo validation; dùng test cho lần báo cáo cuối.

1. Vì sao khoảng cách giữa hai ID từ không có ý nghĩa ngôn ngữ?
2. Trong CBOW, tại sao XW_{in} là trung bình hai hàng embedding?
3. Khi đổi CBOW sang Skip-gram, phần nào của forward và loss vẫn giữ nguyên?
4. Tại sao top-1 accuracy 50% có thể phù hợp với một phân phối CBOW tốt?

Tài liệu tham khảo

[1] T. Mikolov, K. Chen, G. Corrado, J. Dean, *Efficient Estimation of Word Representations in Vector Space*, 2013. https://arxiv.org/abs/1301.3781.

[2] T. Mikolov et al., *Distributed Representations of Words and Phrases and their Compositionality*, 2013. https://arxiv.org/abs/1310.4546.

[3] Trương Xuân Nam, *Xử lý ngôn ngữ tự nhiên*, `slide/NLP-1.pdf`, trang 6–14.

[4] `encoder_decoder_minh_hoa.ipynb`, phần 3–4: dữ liệu, hàm `word_forward`, gradient và kết quả CBOW/Skip-gram.